Taopix
External Customer Account Scripting API

Version: 5.4 – 9th February 2021
Compatible With TAOPIX™ Web v2021r1+

Confidential – Not For Distribution

Revision History

1st November 2013	v2.0	Changed name from External Login Scripting to External Customer Account Scripting
1st November 2013	v2.0	Changed some existing function names to reflect the new purpose of the API. Integrations must be to be modified so that they handle this change of functionality.
1st November 2013	v2.0	Updated to reflect that customer accounts are now unique to the brand. Integrations may need to be modified so that they handle this change of functionality correctly.
1st November 2013	v2.0	Updated the section on limitations and new accounts created, modified or deleted within Taopix are now transmitted to the script. Integrations must be to be modified so that they handle this change of functionality.
1st November 2013	v2.0	Updated the example code to reflect the new API.
1st November 2013	v2.0	Added information on the new *createAccount* API call.
1st November 2013	v2.0	Added information on the new *deleteAccount* API call.
1st November 2013	v2.0	Added information on the new *updateActiveStatus* API call.
1st November 2013	v2.0	Added the columns *id* and *status* to the *login* API call. Integrations must be updated to handle this new column.
1st November 2013	v2.0	Added several new columns and updated the *updateAccountDetails* API call. Integrations must be updated to handle these new columns.
1st November 2013	v2.0	Added the column *id* to the *updatePassword* API call.
1st November 2013	v2.0	Added the column *id* to the *updateAccountPrefs* API call.
1st November 2013	v2.0	Added the column *id* to the *updateAccountBalance* API call.
1st November 2013	v2.0	Added the column *id* to the *updateGiftCardBalance* API call.
1st November 2013	v2.0	Added the column *id* to the *resetPassword* API call.
15th January 2014	v3.0	Added information on the new *loginExists* API call.
15th January 2014	v3.0	Added an *Additional Information* section that contains important information relating to the integration process.
7th June 2018	v4.0	Documentation section added in regards of password sanitisation.
5th September 2018	v5.0	Updated processing reset password requests due to the modernisation of the password reset feature
18th September 2019	V5.1	Added information about ssotoken and ssoprivatedata being returned from login and createAccount functions. Added a note regarding the password value not matching the correct password format when running under HTTPS.
19th September 2019	V5.2	Added usedefaultcurrency and currencycode to the useraccount array as described in the output parameters in the ssoLogin section.
08th October 2019	V5.3	Added method 'authenticate' to re-validate user passwords
9th February 2021	V5.4	Added login field to SSO

Introduction

The Taopix External Customer Account scripting feature is a licensable option within Taopix that provides the following features:

- Allows Taopix to access, create and modify customer account information held within an external system.
- Allows the licensee to integrate with their own login workflow via the Taopix Single Sign-On API functions.

Taopix Licensee Software Development / Skills Required

1. PHP programming language skills to create the custom script.

2. An understanding of how to communicate with the external system holding the customer account information.

3. Basic operational knowledge of Taopix Creator, Taopix Designer, Taopix Web and Taopix Production.

Additional Information

1. Taopix uses UTF-8 as its native text encoding. If the external system does not use UTF-8 it is the responsibility of the integrator to perform any relevant re-encoding when communicating between Taopix and the external system.

2. Taopix stores various names and descriptions for multiple languages in a single string known as the Taopix multi-lingual string format. More information on this subject can be found later within this document.

3. User logins and passwords within Taopix are case sensitive.

IMPORTANT NOTES

1. It is the responsibility of Taopix Limited to provide and support a working API. It will not be the responsibility of Taopix Limited to perform or support any of the integration work required by the licensee.

2. The Taopix system is constantly evolving and it will be necessary for the licensee to update the integration as new features and enhancements are implemented.

Password Sanitisation:

Prior to the Taopix 2018r3 release, passwords passed as input parameters to the new account creation process were previously sanitised. As of 2018r3 release they are no longer sanitised.

Sample Code

The documentation comes complete with an example script that demonstrates some of the basic concepts of the scripting functionality:

1. When logging into Taopix Control Centre the script will check the login details for a login of *edllogintest1* and password *edlpassword1*. If the login matches then it returns a sample user account attached to the license key *EDLDEMO*. If the login does not match then the script returns with an error stating that the account does not exist.

2. When resetting a password the script will check the details for a login of *edllogintest1* and an email address of *edluser@nowhere.com*. If the details match then Taopix will reset the password but not generate a confirmation email. If the details do not match then the script returns with an error stating that the account does not exist.

Sample Code Installation / Configuration

1. Create, configure, upload and enable a license key that has the code *EDLDEMO*.

2. Copy *EDL_ExternalCustomerAccount.php* into *MediaAlbumWeb\customise\scripts.*

NOTE
There is no sample code for implementing Single Sign-On.

Testing The Sample Code

To test the sample code you need to perform, within Taopix, one of the two actions listed above.

Processing New Account Creation Requests

When a Taopix administrator creates a new customer account or when a customer registers a new account, Taopix Control Centre first determines if the account would be valid within the Taopix database. If this validation fails then the account will be rejected as normal. If this validation succeeds, Taopix will divert processing to the `createAccount` function within the external PHP script `EDL_ExternalCustomerAccount.php`.

The purpose of this function is to use the parameters provided as a PHP associative array to create a new customer account within the external system and return the result back to Taopix Control Centre.

```
function createAccount($pParamArray)
{
        $resultArray = Array();
        $result = '';

        $userAccountArray = $pParamArray['useraccount'];
        $newAccountCode = $userAccountArray['accountcode'];

        // perform some processing

        // return the response of the account creation
        $result['result'] = $result;
        $result['newaccountcode'] = $accountCode;

        return $resultArray;
}
```

IMPORTANT
For Single Sign-On, user accounts can be created as a result of the `ssoLogin` function.

Input Parameters

The input parameter array keys provided to the script are as following:

languagecode	varchar(50)	The Taopix language code
isadmin	integer	0 = Customer registering new account 1 = Administrator creating new account
groupcode	varchar(50)	The code for the Taopix License Key that is assigned to the customer account.
brandcode	varchar(50)	The code for the Taopix brand that the customer account is being created for.
accountcode	varchar(50)	The account code for the customer account currently being created.
login	varchar(50)	The login name

passwordformat	integer	The format of the password data:
		`TPX_PASSWORDFORMAT_CLEARTEXT`
		`TPX_PASSWORDFORMAT_MD5`
		
		If Taopix Control Centre is running under HTTPS / SSL then the password will be in clear text otherwise it will be a MD5 hash.

password	varchar(50)	The password in the specified format.
status	integer	The status of the customer account data: -1 = N/A 0 = N/A 1 = An account with all information is being created. 2 = An account with only the basic information is being created from the Taopix Online Designer.
useraccount	array	A PHP associate array that contains the default settings for a new Taopix user account.

The user account array contains the following data:

companyname	varchar(200)	A	Company name
address1	varchar(200)	A	Address line 1
address2	varchar(200)	A	Address line 2
address3	varchar(200)	A	Address line 3
address4	varchar(200)	A	Address line 4
city	varchar(200)	A	City
county	varchar(50)	A	County
state	varchar(200)	A	State
regioncode	varchar(20)	A	Region code
region	varchar(10)	A	Region field code
postcode	varchar(200)	A	Postcode / Zipcode
countrycode	varchar(10)	M	The *iso2* country code that matches a value within the Taopix *COUNTRIES* database table.
countryname	varchar(50)	M	The name of the country that matches the *iso2* country code.
telephonenumber	varchar(50)	M	Telephone number
emailaddress	varchar(50)	M	Email address
contactfirstname	varchar(200)	M	Contact first name
contactlastname	varchar(200)	M	Contact last name
registeredtaxnumbertype	integer	A	The tax number type for the customer. Values greater than 0 are currently only supported for addresses in Brazil. 0 = Not applicable 1 = Personal 2 = Corporate
registeredtaxnumber	varchar(50)	A	The tax number in the format specified by *registeredtaxnumbertype* when *registeredtaxnumbertype* is not 0.
usedefaultpaymentmethods	integer	M	0 = Use custom set of payment methods 1 = Use default payment methods
paymentmethods	varchar(100)	C	A comma separated list of Taopix payment method codes when using *usedefaultpaymentmethods* is 0.
taxcode	varchar(20)	O	A valid Taopix Control Centre tax rate code if applying a custom product tax rate to the customer account or empty if Taopix should determine the product tax rate.
shippingtaxcode	varchar(20)	O	A valid Taopix Control Centre tax rate code if applying a custom shipping tax rate to the customer account or empty if Taopix should determine the shipping tax rate.
defaultaddresscontrol	integer	C	0 = The address / email settings are customised for this user account. 1 = The address / email settings are controlled by the license key.
uselicensekeyforshippingaddress	integer	C	0 = The user account address is used for the shipping address. 1 = The address held in the license key is used for the shipping address. **NOTE** This option only applies if *defaultaddresscontrol* is 0.
canmodifyshippingaddress	integer	C	0 = The shipping address cannot be modified. 1 = The shipping address can be modified. **NOTE** This option only applies if *defaultaddresscontrol* is 0.
canmodifyshippingcontactdetails	integer	C	0 = The shipping address contact details cannot be modified. 1 = The shipping address contact details can be modified.

			NOTE This option only applies if *defaultaddresscontrol* is 0 and *canmodifyshippingaddress* is 0.
uselicensekeyforbillingaddress	integer	C	0 = The user account address is used for the billing address. 1 = The address held in the license key is used for the billing address. **NOTE** This option only applies if *defaultaddresscontrol* is 0.
canmodifybillingaddress	integer	C	0 = The billing address cannot be modified. 1 = The billing address can be modified. **NOTE** This option only applies if *defaultaddresscontrol* is 0.
useremaildestination	integer	C	0 = Send confirmation emails to order billing address. 1 = Send confirmation emails to order shipping address. 2 = Send confirmation emails to order shipping address and bcc order billing address. 3 = Send confirmation emails to order billing address and bcc order shipping address. **NOTE** This option only applies if *defaultaddresscontrol* is 0.
canmodifypassword	integer	M	0 = The password cannot be modified by the user. 1 = The password can be modified by the user.
creditlimit	decimal	O	The credit limit in the currency specified by Taopix as the system currency.
accountbalance	decimal	O	The account balance in the currency specified by Taopix as the system currency.
giftcardbalance	decimal	O	The gift card balance in the currency specified by Taopix as the system currency.
sendmarketinginfo	integer	O	The status of the *Can Send Marketing Information* flag: 0 = No 1 = Yes
isactive	integer	M	0 = Account not active 1 = Account active

Key:

M	Mandatory
O	Optional
C	Conditional Please read the column description for additional information
A	Address field that may be mandatory. Please see the section on address formats for additional information

IMPORTANT

1. The user account array does actually contain more data than listed here. However, it is critical that you always return a modified version of the input array and do not modify any values apart from the ones listed here.

2. Customer accounts are unique across a Taopix brand.

3. It is not possible to have any customer account with the same username / login as any non-customer account.

4. The *isactive* flag has limited use when using external scripts to handle authentication. All customer login requests will be forwarded to the script no matter what is set within the Taopix user account and it is up to the script to determine if the account is active or not. If *isactive* is set to zero in the result Taopix will display the standard message stating that the user account is not active unless an error is returned by the script.

NOTE
Failure to supply Taopix Control Centre with the correct information will result in the system not operating correctly.

Processing The Request

The actual task of creating the customer account within the external system will depend on how the external system actually operates. This is outside of the scope of this document.

Returning The Account Creation Result Back To Taopix

Returning the login result back to Taopix involves the following:
1. Returning a result string which is either empty for a successful creation or an error message that is reported back to the user. The error message must be a Taopix string constant that resides in either the # *Common* or [*AdminCustomers]* / *[Login]* sections of the Taopix language string files depending on what task initiated the request.

2. A string that represents the account code to be stored in the customer account record. The input parameters contain the default value that will be stored and this can simply be returned if a different value is not required (the account code could be used to store a link between the customer accounts residing in different systems).

Output Parameters

result	string	Empty on success or a Taopix language string constant from either the # *Common* or *[AdminCustomers] / [Login]* sections of the Taopix language string files.
accountcode	string	The customer account code to be stored in the database record.
ssotoken	string	If the process of logging the user in resulted in an SSO session starting you can populate this item with any token information you will need. This needs to be populated to indicated that this is now an SSO session.
ssoprivatedata	array	An array of data which will be passed between all SSO related calls. This data is never presented to the front end.

IMPORTANT
1. Customer accounts are unique across a Taopix brand.

2. It is not possible to have any customer account with the same username / login as any non-customer account.

3. Returning an error will also prevent the customer account record from being inserted into the Taopix database.

NOTE
Failure to supply Taopix Control Centre with the correct information will result in the system not operating correctly.

Processing Update Account Details Requests

When an existing customer account is modified, Taopix Control Centre first determines if the account will be valid within the Taopix database. If this validation fails then the modifications will be rejected as normal. If this validation succeeds, Taopix will divert processing to the `updateAccountDetails` function within the external PHP script `EDL_ExternalCustomerAccount.php`.

The purpose of this function is to use the parameters provided as a PHP associative array to find a customer account within the external system, update the external customer account data and return the result back to Taopix Control Centre.

```
function updateAccountDetails($pParamArray)
{
        $result = '';

        // perform some processing

        return $result;
}
```

IMPORTANT
For Single Sign-On, user accounts can be updated as a result of the `ssoLogin` function.

Input Parameters

The input parameter array keys provided to the script are as following:

languagecode	varchar(50)	The Taopix language code
isadmin	integer	0 = Customer modifying account 1 = Administrator modifying account
id	integer	The record id of the customer account record if it already exists within the Taopix database.
origgroupcode	varchar(50)	The code for the Taopix License Key that was originally assigned to the customer account within the Taopix database.
origbrandcode	varchar(50)	The code for the Taopix Brand that was originally assigned to the customer account within the Taopix database.
origaccountcode	varchar(50)	The account code that was originally assigned to the customer account held within the Taopix database.
originlogin	varchar(50)	The login name that was originally assigned to the customer account held within the Taopix database.
newgroupcode	varchar(50)	The code for the Taopix License Key now assigned to the customer account within the Taopix database.
newbrandcode	varchar(50)	The code for the Taopix Brand now assigned to the customer account within the Taopix database.
newaccountcode	varchar(50)	The account code now assigned to the customer account held within the Taopix database.
newlogin	varchar(50)	The login name now assigned to the customer account held within the Taopix database.
passwordchanged	integer	0 = Password has not been changed Note: If the user hasn't changed their password, despite the value in the passwordformat parameter, the password will be hashed as per our database record. 1 = Password has been changed

passwordformat	integer	The format of the password data:
		`TPX_PASSWORDFORMAT_CLEARTEXT`
		`TPX_PASSWORDFORMAT_MD5`
		
		If Taopix Control Centre is running under HTTPS / SSL and the password has been updated then the password will be in clear text otherwise it will be a MD5 hash.

password	varchar(50)	The password in the specified format.

status	integer	The status of the customer account data: -1 = N/A 0 = N/A 1 = An account with all information is being modified. 2 = An account with only the basic information is modified.
useraccount	array	The new settings for the customer account (see below).

The user account array contains the following data:

companyname	varchar(200)	Company name (see address formats below)
address1	varchar(200)	Address line 1 (see address formats below)
address2	varchar(200)	Address line 2 (see address formats below)
address3	varchar(200)	Address line 3 (see address formats below)
address4	varchar(200)	Address line 4 (see address formats below)
city	varchar(200)	City (see address formats below)
county	varchar(50)	County (see address formats below)
state	varchar(200)	State (see address formats below)
regioncode	varchar(20)	Region code (see address formats below)
region	varchar(10)	Region field code (see address formats below)
postcode	varchar(200)	Postcode / Zipcode (see address formats below)
countrycode	varchar(10)	The *iso2* country code that matches a value within the Taopix *COUNTRIES* database table.
countryname	varchar(50)	The name of the country that matches the *iso2* country code.
telephonenumber	varchar(50)	Telephone number
emailaddress	varchar(50)	Email address
contactfirstname	varchar(200)	Contact first name
contactlastname	varchar(200)	Contact last name
registeredtaxnumbertype	integer	The tax number type for the customer. Values greater than 0 are currently only supported for addresses in Brazil. 0 = Not applicable 1 = Personal 2 = Corporate
registeredtaxnumber	varchar(50)	The tax number in the format specified by *registeredtaxnumbertype* when *registeredtaxnumbertype* is not 0.
usedefaultpaymentmethods	integer	0 = Use custom set of payment methods 1 = Use default payment methods
paymentmethods	varchar(100)	A comma separated list of Taopix payment method codes when using *usedefaultpaymentmethods* is 0.
taxcode	varchar(20)	A valid Taopix Control Centre tax rate code if applying a custom product tax rate to the customer account or empty if Taopix should determine the product tax rate.
shippingtaxcode	varchar(20)	A valid Taopix Control Centre tax rate code if applying a custom shipping tax rate to the customer account or empty if Taopix should determine the shipping tax rate.
defaultaddresscontrol	integer	0 = The address / email settings are customised for this user account. 1 = The address / email settings are controlled by the license key.
uselicensekeyforshippingaddress	integer	0 = The user account address is used for the shipping address.

		1 = The address held in the license key is used for the shipping address. **NOTE** This option only applies if *defaultaddresscontrol* is 0.
canmodifyshippingaddress	integer	0 = The shipping address cannot be modified. 1 = The shipping address can be modified. **NOTE** This option only applies if *defaultaddresscontrol* is 0.
canmodifyshippingcontactdetails	integer	0 = The shipping address contact details cannot be modified. 1 = The shipping address contact details can be modified. **NOTE** This option only applies if *defaultaddresscontrol* is 0 and *canmodifyshippingaddress* is 0.
uselicensekeyforbillingaddress	integer	0 = The user account address is used for the billing address. 1 = The address held in the license key is used for the billing address. **NOTE** This option only applies if *defaultaddresscontrol* is 0.
canmodifybillingaddress	integer	0 = The billing address cannot be modified. 1 = The billing address can be modified. **NOTE** This option only applies if *defaultaddresscontrol* is 0.
useremaildestination	integer	0 = Send confirmation emails to order billing address. 1 = Send confirmation emails to order shipping address. 2 = Send confirmation emails to order shipping address and bcc order billing address. 3 = Send confirmation emails to order billing address and bcc order shipping address. **NOTE** This option only applies if *defaultaddresscontrol* is 0.
canmodifypassword	integer	0 = The password cannot be modified by the user. 1 = The password can be modified by the user.
creditlimit	decimal	The credit limit in the currency specified by Taopix as the system currency.
accountbalancedifference	decimal	The account balance difference in the currency specified by Taopix as the system currency.
giftcardbalancedifference	decimal	The gift card balance difference in the currency specified by Taopix as the system currency.
sendmarketinginfo	integer	The status of the *Can Send Marketing Information* flag: 0 = No 1 = Yes
isactive	integer	0 = Account not active 1 = Account active

Processing An Update Account Details Request

The actual task of updating the account details will depend upon the external system responsible for the customer accounts but it will probably involve checking the brand code and login supplied by Taopix and updating the customer account held within the external system with the new details.

Returning The Update Account Details Request Result Back To Taopix

Returning the update account details request result back to Taopix involves returning a string that is either empty for a successful update or an error message that is reported back to the user. The error message must be a Taopix string constant that resides in either the # *Common* or [*AdminCustomers]* / *[Login]* sections of the Taopix language string files depending on what task initiated the request.

IMPORTANT
Returning an error will also prevent the customer account record from being modified within the Taopix database.

NOTE
Failure to supply Taopix Control Centre with the correct information will result in the system not operating correctly.

Processing Account Activation And Deactivation Requests

When a Taopix administrator either activates or deactivates a customer account the script must handle this request. Taopix Control Centre will first attempt to perform the task for the specific record within the Taopix database and if that fails the process will be aborted. If the process succeeds Taopix will divert processing to the `updateActiveStatus` function within the external PHP script `EDL_ExternalCustomerAccount.php`.

The purpose of this function is to use the parameters provided as a PHP associative array to find a customer account within the external system, update the active status of the account and return the result back to Taopix Control Centre.

```
function updateActiveStatus($pParamArray)
{
        $result = '';

        // perform some processing

        // return the response of the request

        return $result;
}
```

Input Parameters

The input parameter array keys provided to the script are as following:

languagecode	varchar(50)	The Taopix language code
groupcode	varchar(50)	The code for the Taopix License Key that is assigned to the customer account within the Taopix database.
brandcode	varchar(50)	The code for the Taopix Brand that is assigned to the customer account within the Taopix database.
accountcode	varchar(50)	The account code that is assigned to the customer account held within the Taopix database.
id	integer	The record id of the customer account record if it already exists within the Taopix database.
login	varchar(50)	The login name
isactive	integer	0 = The customer account needs to be deactivated. 1 = The customer account needs to be activated.

Processing A Change Active Status Request

The actual task of changing the active status will depend upon the external system responsible for the customer accounts and is out of the scope of this document.

Returning The Change Active Status Request Result Back To Taopix

Returning the update active status request result back to Taopix involves returning a result string that is either empty for a successful request or an error message that is reported back to the user. The error message must be a Taopix string constant that resides in either the *# Common* or *[AdminCustomers]* sections of the Taopix language string files.

IMPORTANT
Returning an error will also prevent the customer account record from being modified within the Taopix database.

Processing Account Deletion Requests

When a Taopix administrator deletes a customer account the script must handle this request. Taopix Control Centre will first attempt to perform the task for the specific record within the Taopix database and if that fails the process will be aborted. If the process succeeds Taopix will divert processing to the `deleteAccount` function within the external PHP script `EDL_ExternalCustomerAccount.php`.

The purpose of this function is to use the parameters provided as a PHP associative array to find a customer account within the external system, delete it and return the result back to Taopix Control Centre.

```
function deleteAccount($pParamArray)
{
        $result = '';

        // perform some processing

        // return the response of the request

        return $result;
}
```

Input Parameters

The input parameter array keys provided to the script are as following:

languagecode	varchar(50)	The Taopix language code
groupcode	varchar(50)	The code for the Taopix License Key that is assigned to the customer account within the Taopix database.
brandcode	varchar(50)	The code for the Taopix Brand that is assigned to the customer account within the Taopix database.
accountcode	varchar(50)	The account code that is assigned to the customer account held within the Taopix database.
id	integer	The record id of the customer account record if it already exists within the Taopix database.
login	varchar(50)	The login name

Processing A Delete Account Request

The actual task of deleting a customer account will depend upon the external system responsible for the customer accounts and is out of the scope of this document.

Returning The Delete Account Request Result Back To Taopix

Returning the delete account request status result back to Taopix involves returning a result string that is either empty for a successful request or an error message that is reported back to the user. The error message must be a Taopix string constant that resides in either the *# Common* or *[AdminCustomers]* sections of the Taopix language string files.

IMPORTANT
Returning an error will also prevent the customer account record from being deleted within the Taopix database.

Processing Reset Password Requests

If a customer is allowed to reset their password via the Taopix login page then the script must handle this request. When Taopix Control Centre receives the request it first determines if a user account with the details is held within the Taopix database. If a user account is found and the details are for a non-customer account then Taopix will handle the request as normal. If a user account is found and the details are for a customer account or if no user account is found then Taopix will divert processing to the `resetPasswordInit` function within the external PHP script `EDL_ExternalCustomerAccount.php`.

In 2018r5 the password reset feature was modernised so that a password reset link is now emailed to the customer to allow them to create a new password. The purpose of this function is to determine whether or not Taopix should handle this reset password request or should the external system handle it.

If Taopix is to handle the request, a reset password email will be generated which contains the reset link to redirect the user to Taopix Control Centre in order to complete the process. This function allows the external system to by-pass the Taopix workflow and perform its own password reset process.

```
function resetPasswordInit($pParamArray)
{
        $resultArray = Array();
        $result = 'NOTHANDLED';
        $redirectURL = '';

        // perform some processing

        // return the response of the request
        $result['result'] = $result;
        $result['redirecturl'] = $redirectURL;

        return $resultArray;
}
```

IMPORTANT
This function does not need implementing for Single Sign-On integrations since user passwords are not stored in Control Centre for Single Sign-On users.

Input Parameters

The input parameter array keys provided to the script are as following:

languagecode	varchar(50)	The Taopix language code
designergroupcode	varchar(50)	The code for the Taopix License Key that was supplied to Taopix Control Centre by Taopix Designer.
accountgroupcode	varchar(50)	The code for the Taopix License Key that is assigned to the customer account if one already exists within the Taopix database.
brandcode	varchar(50)	The code for the Taopix brand that the user is browsing from.
accountcode	varchar(50)	The account code for the user account held within the Taopix database that matches the login and email address.
id	integer	The record id of the customer account record if it already exists within the Taopix database.
login	varchar(50)	The login name
emailaddress	varchar(50)	The email address
isordersession	integer	0 = The reset request came when the user was not about to place an order. 1 = The reset request came when the user was about to place an order.

Processing A Reset Password Request

The actual task of resetting a password will depend upon the external system responsible for the customer accounts but it will probably involve checking the brand code, login and email address supplied by Taopix to find a matching customer account held within the external system and determine if it should handle the password reset request or should Taopix.

Returning The Reset Password Request Result Back To Taopix

Returning the reset password request result back to Taopix involves the following:

1. Returning a result string that is either NOTHANDLED for Taopix to handle the password reset process or REDIRECT for the external system to handle the password reset. An error message can also be returned back to the user. The error message must be a Taopix string constant that resides in either the # Common or [Login] sections of the Taopix language string files.
2. Returning a redirecturl along with a result of REDIRECT will inform Taopix that a user should be redirected to the URL provided in order to complete the reset password process. If the redirecturl is returned as empty and the result is REDIRECT this indicates that the external system will handle the reset password process but Taopix should display the confirmation screen to the user that states that a reset email has been sent.

Output Parameters

result	string	NOTHANDLED – Taopix to handle the password reset request. REDIRECT – External system will handle the password reset request. Anything else returned should be a Taopix language string constant from either the # *Common* or *[Login]* sections of the Taopix language string files.
redirecturl	string	A valid URL to redirect the customer so they can complete the password reset request in the external system.

Processing A Reset Password Request From The Reset Password Email Link

If Taopix handled the password reset request then the user would have received an email containing a link that redirects them to Taopix Control Centre where they can change their password and complete their reset request.

When Taopix Control Centre receives the request to create the new password it first determines if a user account with the details is held within the Taopix database. If a user account is found and the details are for a non-customer account then Taopix will handle the request as normal. If a user account is found and the details are for a customer account or if no user account is found then Taopix will divert processing to the `resetPassword` function within the external PHP script `EDL_ExternalCustomerAccount.php`.

The purpose of this function is to use the parameters provided as a PHP associative array to find a customer account within the external system, reset the external customer account password based of the new password the user entered and return the relevant data, also as an associative array, back to Taopix Control Centre.

```
function resetPassword($pParamArray)
{
        $resultArray = Array();
        $result = '';
        $sendNotification = false;
        $storePassword = false;
        $contactFirstName = '';
        $contactLastName = '';

        // perform some processing

        // return the response of the request
        $result['result'] = $result;
        $result['sendnotification'] = $sendNotification;
        $result['storepassword'] = $storePassword;
        $result['contactfirstname'] = $contactFirstName;
        $result['contactlastname'] = $contactLastName;

        return $resultArray;
}
```

IMPORTANT
This function does not need implementing for Single Sign-On integrations since user passwords are not stored in Control Centre for Single Sign-On users.

Input Parameters

The input parameter array keys provided to the script are as following:

languagecode	varchar(50)	The Taopix language code
designergroupcode	varchar(50)	The code for the Taopix License Key that was supplied to Taopix Control Centre by Taopix Designer.
accountgroupcode	varchar(50)	The code for the Taopix License Key that is assigned to the customer account if one already exists within the Taopix database.
brandcode	varchar(50)	The code for the Taopix brand that the user is browsing from.
accountcode	varchar(50)	The account code for the user account held within the Taopix database that matches the login and email address.
id	integer	The record id of the customer account record if it already exists within the Taopix database.
login	varchar(50)	The login name
emailaddress	varchar(50)	The email address
newpassword	varchar(50)	The new password in the specified format.

passwordformat	integer	The format of the password data:
		`TPX_PASSWORDFORMAT_CLEARTEXT`
		`TPX_PASSWORDFORMAT_MD5`
		
		If Taopix Control Centre is running under HTTPS / SSL then the password will be in clear text otherwise it will be a MD5 hash.

isordersession	integer	0 = The reset request came when the user was not about to place an order. 1 = The reset request came when the user was about to place an order.

Processing A Reset Password Request

The actual task of resetting a password will depend upon the external system responsible for the customer accounts but it will probably involve checking the brand code, login and email address supplied by Taopix to find a matching customer account held within the external system and then update the external customer account with the users new chosen password.

Returning The Reset Password Request Result Back To Taopix

Returning the reset password request result back to Taopix involves the following:
1. Returning a result string that is either empty for a successful request or an error message that is reported back to the user. The error message must be a Taopix string constant that resides in either the # *Common* or *[Login]* sections of the Taopix language string files.

2. A flag specifying if Taopix Control Centre should generate the password reset confirmation email.

3. A flag specifying if the users password can be updated within the Taopix database.

4. The customer's name and new password if Taopix Control Centre is going to generate the password reset notification email.

Output Parameters

result	string	Empty on success or a Taopix language string constant from either the # *Common* or *[Login]* sections of the Taopix language string files.
sendnotification	boolean	false = Taopix Control Centre will not send the password reset confirmation email. true = Taopix Control Centre will send the password reset confirmation email.
storepassword	boolean	false = The customer account held within the Taopix database will not be updated with the new password. true = The customer account held within the Taopix database will be updated with the new password.
contactfirstname	string	The customer's contact first name if Taopix Control Centre is sending the password reset confirmation email otherwise an empty string.
contactlastname	string	The customer's contact last name if Taopix Control Centre is sending the password reset confirmation email otherwise an empty string.

Processing Password Update Requests

If a customer is allowed to sign-in to their account within Taopix and has access to the password update feature then the script must handle this request. When Taopix Control Centre receives the request it will divert processing to the `updatePassword` function within the external PHP script `EDL_ExternalCustomerAccount.php`.

The purpose of this function is to use the parameters provided as a PHP associative array to find a customer account within the external system, update the external customer account password and return the result back to Taopix Control Centre.

```
function updatePassword($pParamArray)
{
        $result = '';

        // perform some processing

        return $result;
}
```

IMPORTANT
This function does not need implementing for Single Sign-On integrations since user passwords are not stored in Control Centre for Single Sign-On users.

Input Parameters

The input parameter array keys provided to the script are as following:

languagecode	varchar(50)	The Taopix language code
groupcode	varchar(50)	The code for the Taopix License Key assigned to the customer account within the Taopix database.
brandcode	varchar(50)	The code for the Taopix brand that the user is browsing from.
accountcode	varchar(50)	The account code for the customer account held within the Taopix database.
id	integer	The record id of the customer account record if it already exists within the Taopix database.
login	varchar(50)	The login name

passwordformat	integer	The format of the password data:
		`TPX_PASSWORDFORMAT_CLEARTEXT`
		`TPX_PASSWORDFORMAT_MD5`
		
		If Taopix Control Centre is running under HTTPS / SSL then the password will be in clear text otherwise it will be a MD5 hash.

origpassword	varchar(50)	The original password in the specified format.
newpassword	varchar(50)	The new password in the specified format.

Processing An Update Password Request

The actual task of updating a password will depend upon the external system responsible for the customer accounts but it will probably involve checking the brand code, login and original password supplied by Taopix and updating the customer account held within the external system with the new password.

Returning The Update Password Request Result Back To Taopix

Returning the update password request result back to Taopix involves returning a string that is either empty for a successful update or an error message that is reported back to the user. The error message must be a Taopix string constant that resides in either the *# Common* or *[Customer]* sections of the Taopix language string files.

Processing Update Account Preferences Requests

If a customer is allowed to sign-in to their account within Taopix then the script must handle the request to update their account preferences. When Taopix Control Centre receives the request it will divert processing to the `updateAccountPrefs` function within the external PHP script `EDL_ExternalCustomerAccount.php`.

The purpose of this function is to use the parameters provided as a PHP associative array to find a customer account within the external system, update the external account preferences and return the result back to Taopix Control Centre.

```
function updateAccountPrefs($pParamArray)
{
        $result = '';

        // perform some processing

        return $result;
}
```

IMPORTANT
This function does not need implementing for Single Sign-On integrations since user passwords are not stored in Control Centre for Single Sign-On users.

Input Parameters

The input parameter array keys provided to the script are as following:

languagecode	varchar(50)	The Taopix language code
groupcode	varchar(50)	The code for the Taopix License Key assigned to the customer account within the Taopix database.
brandcode	varchar(50)	The code for the Taopix brand that the user is browsing from.
accountcode	varchar(50)	The account code for the customer account held within the Taopix database.
id	integer	The record id of the customer account record if it already exists within the Taopix database.
login	varchar(50)	The login name
sendmarketinginfo	integer	The new status of the *Can Send Marketing Information* flag: 0 = No 1 = Yes

Processing An Update Account Preferences Request

The actual task of updating the account preferences will depend upon the external system responsible for the customer accounts but it will probably involve checking the brand code and login supplied by Taopix and updating the customer account held within the external system with the new preferences.

Returning The Update Account Preferences Request Result Back To Taopix

Returning the update account preferences request result back to Taopix involves returning a string which is either empty for a successful update or an error message that is reported back to the user. The error message must be a Taopix string constant that resides in either the # *Common* or *[Customer]* sections of the Taopix language string files.

Processing Update Account Balance Requests

If the *ACCOUNT* payment method is used then the script must handle the request to update a customer's account balance when an order has been placed using this payment method. When the *Account* payment method is used the Taopix shopping cart will perform it's normal actions and will then divert processing to the `updateAccountBalance` function within the external PHP script `EDL_ExternalCustomerAccount.php`.

The purpose of this function is to use the parameters provided as a PHP associative array to find the correct customer account within the external system and then update its account balance.

```
function updateAccountBalance($pParamArray)
{
        // perform some processing
}
```

IMPORTANT
For Single Sign-On, user account balances can be updated as a result of the `ssoLogin` function.

Input Parameters

The input parameter array keys provided to the script are as following:

languagecode	varchar(50)	The Taopix language code
groupcode	varchar(50)	The code for the Taopix License Key assigned to the customer account within the Taopix database.
brandcode	varchar(50)	The code for the Taopix Brand that the user is browsing from.
accountcode	varchar(50)	The account code for the customer account held within the Taopix database.
id	integer	The record id of the customer account record if it already exists within the Taopix database.
login	varchar(50)	The login name.
accountbalancedifference	decimal	The amount to increase the account balance by.

Processing An Update Account Balance Request

The actual task of updating the account balance will depend upon the external system responsible for the customer accounts but it will probably involve checking the brand code and login supplied by Taopix and updating the balance of the customer account held within the external system.

Processing Update Gift Card Balance Requests

If the gift card feature is used within Taopix then the script must handle the request to update a customer's gift card balance. When a gift card is redeemed or the gift card balance is used during payment of an order the Taopix shopping cart will perform it's normal actions and will then divert processing to the `updateGiftCardBalance` function within the external PHP script `EDL_ExternalCustomerAccount.php`.

The purpose of this function is to use the parameters provided as a PHP associative array to find the correct customer account within the external system and update its gift card balance.

```
function updateGiftCardBalance($pParamArray)
{
        // perform some processing
}
```

IMPORTANT
For Single Sign-On, user gift card balances can be updated as a result of the `ssoLogin` function.

Input Parameters

The input parameter array keys provided to the script are as following:

languagecode	varchar(50)	The Taopix language code
groupcode	varchar(50)	The code for the Taopix License Key assigned to the customer account within the Taopix database.
brandcode	varchar(50)	The code for the Taopix Brand that the user is browsing from.
accountcode	varchar(50)	The account code for the customer account held within the Taopix database.
id	integer	The record id of the customer account record if it already exists within the Taopix database.
login	varchar(50)	The login name.
isordersession	integer	0 = Customer is not currently placing an order 1 = Customer is currently placing an order
giftcardbalancedifference	decimal	The amount to increase / decrease the gift card balance by.

Processing An Update Gift Card Balance Request

The actual task of updating the gift card balance will depend upon the external system responsible for the customer accounts but it will probably involve checking the brand code and login supplied by Taopix and updating the gift card balance of the customer account held within the external system.

Processing Login Requests

When a user enters their login and password Taopix Control Centre first determines if a user account with the details is held within the Taopix database. If a user account is found and the details are for a non-customer account then Taopix will handle the login as normal. If a user account is found and the details are for a customer account or if no account is found at all then Taopix will divert processing to the `login` function within the external PHP script `EDL_ExternalCustomerAccount.php`.

The purpose of this function is to use the parameters provided as a PHP associative array to find a customer account within the external system and return the relevant data, also as an associative array, back to Taopix Control Centre.

```
function login($pParamArray)
{
        $resultArray = Array();
        $result = '';
        $updateAccountBalance = false;
        $updateGiftCardBalance = false;

        $userAccountArray = $pParamArray['useraccount'];

        // perform some processing

        // return the response of the external login
        $result['result'] = $result;
        $result['useraccount'] = $userAccountArray;
        $result['updateaccountbalance'] = $updateAccountBalance;
        $result['updategiftcardbalance'] = $updateGiftCardBalance;

        return $resultArray;
}
```

IMPORTANT
This function is used when a user attempts to sign-in manually. Use the `ssoLogin` function for processing a login with Single Sign-On.

Input Parameters

The input parameter array keys provided to the script are as following:

languagecode	varchar(50)	The Taopix language code
designergroupcode	varchar(50)	The code for the Taopix License Key that was supplied to Taopix Control Centre by Taopix Designer.
accountgroupcode	varchar(50)	The code for the Taopix License Key that is assigned to the customer account if one already exists within the Taopix database.
brandcode	varchar(50)	The code for the Taopix brand that the user is logging in from.
accountcode	varchar(50)	The account code for the customer account currently held within the Taopix database that matches the login details.
id	integer	The record id of the customer account record if it already exists within the Taopix database.
login	varchar(50)	The login name

passwordformat	integer	The format of the password data:
		`TPX_PASSWORDFORMAT_CLEARTEXT`
		`TPX_PASSWORDFORMAT_MD5`
		
		If Taopix Control Centre is running under HTTPS / SSL then the password will be in clear text otherwise it will be a MD5 hash.

password	varchar(50)	The password in the specified format.
status	integer	The status of the -1 = An account does not exist in the Taopix database 0 = An account exists in the Taopix database and Taopix suggests that the customer be prompted to check their details. 1 = An account exists in the Taopix database

		2 = An account with only the basic information exists within the Taopix database and Taopix suggests that the customer be prompted for the remaining information.
isordersession	integer	0 = User is logging into their account 1 = User is logging in to place an order
useraccount	array	A PHP associate array that contains the default settings for a new Taopix user account.

NOTES
Depending on where the login originated and the data held within the Taopix database the *designergroupcode* and *accountgroupcode* keys will contain different values:
1. If the login originates from a user placing an order originating from Taopix Designer then *designergroupcode* will contain the code that was set in the license key that is being used by Taopix Designer.

2. If the login originates from a user placing a re-order or a social sharing order then *designergroupcode* will contain the code that was set when the original order was placed.

3. If the login originates when not ordering *designergroupcode* will be empty.

4. The *accountgroupcode* value will depend upon if a customer account with the login already exists within the Taopix database. If a customer account does exist then *accountgroupcode* will contain the license key code that the customer account is linked to. If a customer account does not exist then *accountgroupcode* will be empty.

Processing A Login Request

The actual task of finding a customer account will depend upon the external system responsible for the customer accounts but it will probably involve checking the brand code, login and password supplied by Taopix.

Returning The Login Result Back To Taopix

Returning the login result back to Taopix involves the following:
1. Returning a result string which is either:
   a) An empty string for a successful authentication.

   b) The text *NOTHANDLED* if the script did not process the request and Taopix should handle the login as if the script did not execute.

   c) An error message that is reported back to the user. The error message must be a Taopix string constant that resides in either the # *Common* or *[Login]* sections of the Taopix language string files.

2. A copy of the input user account array that has the relevant fields filled in via the script. The array key values are not relevant if the authentication is not successful.

3. A flag that specifies if the account balance / credit limit supplied by the script should override the values currently held within the Taopix database customer account if the authentication is successful.

4. A flag that specifies if the gift card balance supplied by the script should override the values currently held within the Taopix database customer account if the authentication is successful.

Output Parameters

result	string	Empty on success, the text *NOTHANDLED* or a Taopix language string constant from either the # *Common* or *[Login]* sections of the Taopix language string files.
useraccount	array	The modified input useraccount array that contains the settings for the Taopix user account (see below).
updateaccountbalance	boolean	false = Do not update the credit limit / account balance values currently held in the Taopix database user account with the values returned by the script. true = Replace the credit limit / account balance values currently held in the Taopix database user account with the values returned by the script.
updategiftcardbalance	boolean	false = Do not update the gift card balance value currently held in the Taopix database user account with the value returned by the script. true = Replace the gift card balance value currently held in the Taopix database user account with the value returned by the script.
ssotoken	string	If the process of logging the user in resulted in an SSO session starting you can populate this item with any token information you will need. This needs to be populated to indicated that this is now an SSO session.
ssoprivatedata	array	An array of data which will be passed between all SSO related calls. This data is never presented to the front end.

The user account array contains the following data:

groupcode	varchar(50)	M	The code for a valid Taopix License Key that should be assigned to the user account. **NOTE** If you are allowing customers to sign-in to their account within Taopix when not placing an order this value cannot be retrieved from the *designergroupcode* input value and must be sourced from somewhere else. If the only source for the group code is Taopix Designer then the script must prevent the ability to sign-in when *designergroupcode* is empty.
accountcode	varchar(50)	O	The account code for the external user account
companyname	varchar(200)	A	Company name
address1	varchar(200)	A	Address line 1
address2	varchar(200)	A	Address line 2
address3	varchar(200)	A	Address line 3
address4	varchar(200)	A	Address line 4
city	varchar(200)	A	City
county	varchar(50)	A	County
state	varchar(200)	A	State
regioncode	varchar(20)	A	Region code
region	varchar(10)	A	Region field code
postcode	varchar(200)	A	Postcode / Zipcode
countrycode	varchar(10)	M	The *iso2* country code that matches a value within the Taopix *COUNTRIES* database table.
countryname	varchar(50)	M	The name of the country that matches the *iso2* country code.
telephonenumber	varchar(50)	M	Telephone number
emailaddress	varchar(50)	M	Email address
contactfirstname	varchar(200)	M	Contact first name
contactlastname	varchar(200)	M	Contact last name
registeredtaxnumbertype	integer	A	The tax number type for the customer. Values greater than 0 are currently only supported for addresses in Brazil. 0 = Not applicable 1 = Personal 2 = Corporate
registeredtaxnumber	varchar(50)	A	The tax number in the format specified by *registeredtaxnumbertype* when *registeredtaxnumbertype* is not 0.
usedefaultpaymentmethods	integer	M	0 = Use custom set of payment methods 1 = Use default payment methods

paymentmethods	varchar(100)	C	A comma separated list of Taopix payment method codes when using *usedefaultpaymentmethods* is 0.
taxcode	varchar(20)	O	A valid Taopix Control Centre tax rate code if applying a custom product tax rate to the customer account or empty if Taopix should determine the product tax rate.
shippingtaxcode	varchar(20)	O	A valid Taopix Control Centre tax rate code if applying a custom shipping tax rate to the customer account or empty if Taopix should determine the shipping tax rate.
defaultaddresscontrol	integer	C	0 = The address / email settings are customised for this user account. 1 = The address / email settings are controlled by the license key.
uselicensekeyforshippingaddress	integer	C	0 = The user account address is used for the shipping address. 1 = The address held in the license key is used for the shipping address. **NOTE** This option only applies if *defaultaddresscontrol* is 0.
canmodifyshippingaddress	integer	C	0 = The shipping address cannot be modified. 1 = The shipping address can be modified. **NOTE** This option only applies if *defaultaddresscontrol* is 0.
canmodifyshippingcontactdetails	integer	C	0 = The shipping address contact details cannot be modified. 1 = The shipping address contact details can be modified. **NOTE** This option only applies if *defaultaddresscontrol* is 0 and *canmodifyshippingaddress* is 0.
uselicensekeyforbillingaddress	integer	C	0 = The user account address is used for the billing address. 1 = The address held in the license key is used for the billing address. **NOTE** This option only applies if *defaultaddresscontrol* is 0.
canmodifybillingaddress	integer	C	0 = The billing address cannot be modified. 1 = The billing address can be modified. **NOTE** This option only applies if *defaultaddresscontrol* is 0.
useremaildestination	integer	C	0 = Send confirmation emails to order billing address. 1 = Send confirmation emails to order shipping address. 2 = Send confirmation emails to order shipping address and bcc order billing address. 3 = Send confirmation emails to order billing address and bcc order shipping address. **NOTE** This option only applies if *defaultaddresscontrol* is 0.
canmodifypassword	integer	M	0 = The password cannot be modified by the user.

			1 = The password can be modified by the user.
creditlimit	decimal	O	The credit limit in the currency specified by Taopix as the system currency.
accountbalance	decimal	O	The account balance in the currency specified by Taopix as the system currency.
giftcardbalance	decimal	O	The gift card balance in the currency specified by Taopix as the system currency.
sendmarketinginfo	integer	O	The status of the *Can Send Marketing Information* flag: 0 = No 1 = Yes
isactive	integer	M	0 = Account not active 1 = Account active

Key:

M	Mandatory
O	Optional
C	Conditional Please read the column description for additional information
A	Address field that may be mandatory. Please see the section on address formats for additional information

IMPORTANT

1. The user account array does actually contain more data than listed here. However, it is critical that you always return a modified version of the input array and do not modify any values apart from the ones listed here.

2. Customer accounts are unique across a Taopix brand.

3. It is not possible to have any customer account with the same username / login as any non-customer account.

4. The *isactive* flag has limited use when using external scripts to handle authentication. All customer login requests will be forwarded to the script no matter what is set within the Taopix user account and it is up to the script to determine if the account is active or not. If *isactive* is set to zero in the result Taopix will display the standard message stating that the user account is not active unless an error is returned by the script.

NOTE

Failure to supply Taopix Control Centre with the correct information will result in the system not operating correctly.

Processing Login Exists Requests

The login for a non-customer account must be unique within the Taopix database. This means that it cannot already exist as a customer account or another non-customer account. When Taopix is used in conjunction with an external system that is at least partially responsible for customer accounts any non-customer account logins must not exist as customer accounts within the external system.

When an administrator attempts to create a new non-customer account or attempts to modify the login for an existing non-customer account within Taopix Control Centre it will first divert processing to the `loginExists` function within the external PHP script `EDL_ExternalCustomerAccount.php`.

The purpose of this function is to use the parameters provided as a PHP associative array to determine if the login exists within the external system and return the result back to Taopix Control Centre.

```
function loginExists($pParamArray)
{
        $result = false;

        // perform some processing

        return $result;
}
```

Input Parameters

The input parameter array keys provided to the script are as following:

id	integer	The record id of the customer account record if it already exists within the Taopix database.
login	varchar(50)	The login name

Processing A Login Exists Request

The actual task of checking if the login exists will depend upon the external system responsible for the customer accounts but it will involve comparing the login with all customer account logins held within the external system.

Returning The Login Exists Request Result Back To Taopix

Returning the result back to Taopix involves returning a Boolean value which is false if the login does not exist in the external system and it is safe for Taopix to continue or true if the login does exist and Taopix should report an error back to the user.

Taopix Single Sign-On

Single Sign-On (SSO) allows a licensee to by-pass the Taopix login workflow and interact with their own authentication system to provide user access security. It can be invoked 2 ways; Via a server to server call or redirecting the user to a different site. Performing a server to server call is relatively straight forward, all you need to do return the relevant data from the `ssoLogin` function as explained later on in this section. Redirecting the user to a different site to perform the login is a little more involved. Again the section on "Single Sign-On Login" explains how to implement this, I will now attempt to generalise the process.

When `ssoLogin` has been called and you have specified the URL you wish the user to be taken to (using *SSOREDIRECT* in the result variable) we will store the URL of the original call i.e. If the user was attempting to create an online project, we would store the URL for the `createProject` call. This is tagged with a key which is stored in a cookie. The user is then redirected to the URL you have specified to perform the login. Once your system is happy that the login is a success you will need to redirect the user back to the single sign-on callback URL.

This URL is will be:

```
http(s)://<web_url>/sso/callback.php
```

So if the Web URL for your brand is http://www.joebloggsphotos.com the URL to callback will be:

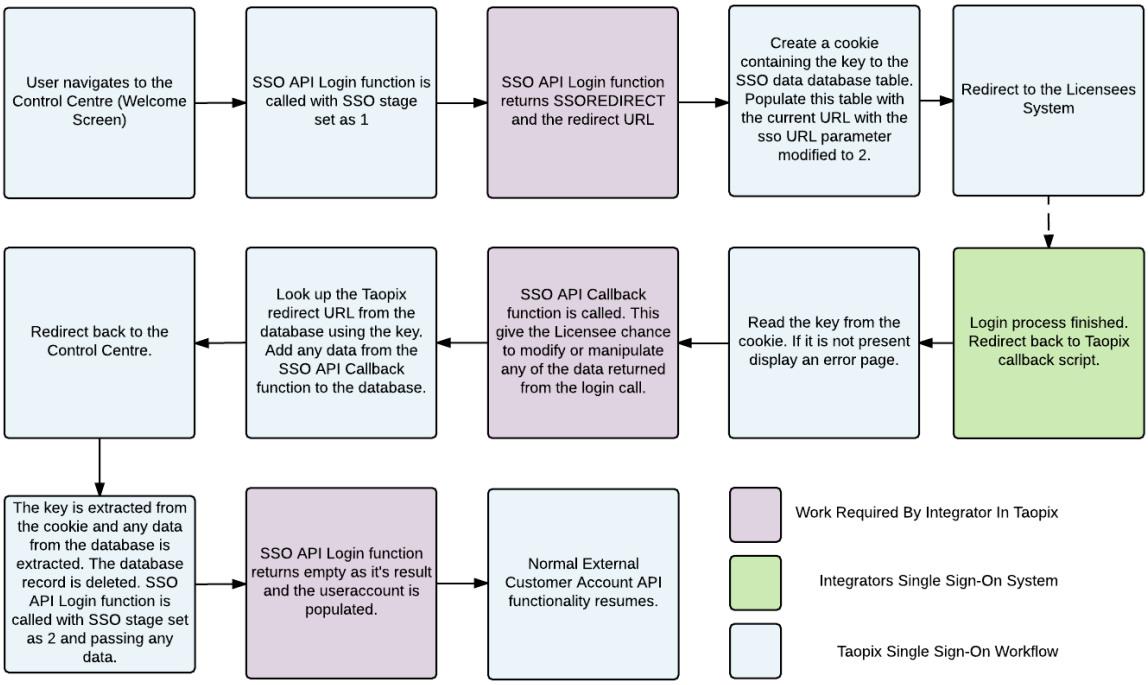
```
http://www.joebloggsphotos.com/sso/callback.php
```

Once the callback script is called the `ssoCallback` function is invoked (explained later on) and the original URL is invoked again. Since the original URL is invoked again the `ssoLogin` function will be called again too. So that you can determine the difference between the two calls one of the parameters passed into the function is the *ssostage*; 1 = first time called, 2 = second time called. The *ssostage* will only every be set to 1 if you are not redirecting to another site for your login workflow since the URL will only be called the once. When the function is called for the second time, any values which have been initialised in the `ssoCallback` function will now be present in the appropriate variables in this function.
The user record and *ssotoken* will need to be populated when the `ssoLogin` function is called under these circumstances:

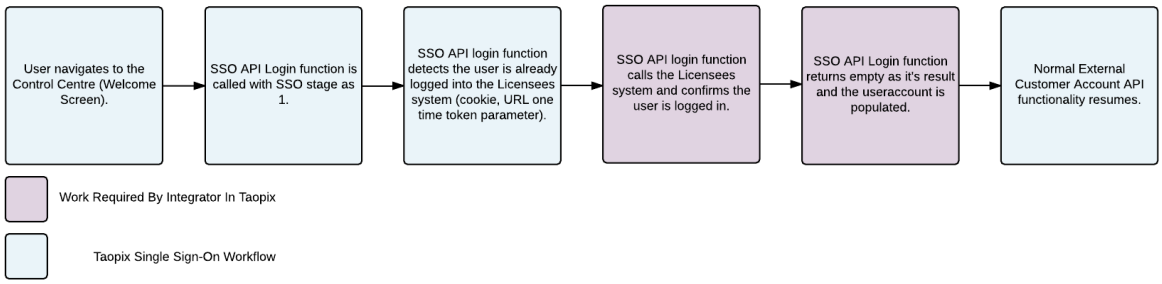
1. The *ssostage* is set to 2. This will only be if the Single Sign-On workflow required the user to be redirected.

2. The *ssostage* is set to 1 and the Single Sign-On workflow doesn't require the user to be redirected.

The following diagram is an example of how the Single Sign-On process could flow if the user is not logged in to the external system:



The following diagram is an example of how the Single Single-On process could flow if the user is already logged in to the external system:



Single Sign-On has been implemented by adding extra functions have been added to the Taopix External Customer Account Scripting API:

`ssoLogin` – This is the main function which needs implementing. This function will be invoked whenever and action is performed in Taopix Control Centre which requires authenticating.

`ssoGetBrandDefaultGroupCode` – Before calling `ssoLogin` Taopix checks to see if a default group code has been set, if not this function will be invoked (if defined). This gives the licensee the chances to set the group code if it is needed.

`ssoLogout` – This function is invoked when a logout function is called from Taopix Control Centre or Taopix Online. This allows you to perform you own logout work flow.

`ssoCallback` – If you have performed your login workflow by redirecting the user to a different site, on returning to Taopix this function is called.

`ssoSetHomeURL` – When an email for a user is generated from Taopix a link to their user account page is usually attached. This function allows you to add extra parameters to the link which might be needed for your Single Sign-On workflow to work.

ssoToken and ssoPrivateData

Throughout this section on Single Sign-On we will talk a lot about the *ssoToken* and *ssoPrivateData*. These are 2 variables which are persistent to the Taopix session which has been created and reside with the Taopix session data.

The *ssoToken* variable is a variable which allows you to store any token information you might want to make visible to Taopix which identifies your session with your system. This data is never read by Taopix code but is stored against the session record and is provided to the Taopix Online system upon creating or editing a project. This token is also passed on to the Library Link API to allow for Single Sign-On authentication to work for any asset management systems. The *ssoToken* will be displayed in Javascript code so make sure the data within is not sensitive data which should not be exposed.
Currently we do not support the the *ssoToken* expiring so it is important to make sure that the lifespan of the token is unlimited or you are able to increase it when needed.

The *ssoPrivateData* is a PHP array which will be provide to the SSO functions like the *ssoToken* is. The variable is stored with the Taopix session data but not in a separate column unlike the *ssoToken* is. The *ssoPrivateData* is designed to store sensitive data which will never find its way into HTML or Javascript. The `ssoCallback` function utilities this variable for storing data from the callback into Taopix for usage later on, which we will talk about this later on.

NOTE
If your Taopix Control Centre system is not SSL and the system you are redirecting the user to login is SSL, when performing the redirect to the callback URL your user could be warned by their browser of insecure data transfers. It is recommended that both systems are running under SSL.

Brand Settings

When you create a new brand a directory called `sso` is created in your brand directory. This will contain two files; `callback.php` and `cookie.php`. These files point to files in the `webroot/sso` directory with the same names. If you already have a brand installed and you wish to implement SSO, you will need to create this directory and install the files manually.

1. Go to the `MediaAlbumWeb/templates` directory.
2. Copy the `sso` directory.
3. Got to the `MediaAlbumWeb/webroot/Branding/<brand>` directory.
4. Paste the `sso` directory in here.

You will need to repeat the above steps for each of your brands.

Example Code

Since Single Sign-On involves interacting with an external systems workflow it is extremely difficult for Taopix to provide any example code to this section of the API. Instead we will give you some hints on how you could implement the various functions.

Single Sign-On Login

When a user attempts to invoke Taopix functionality which needs authentication the `ssoLogin` function within the external PHP script `EDL_ExternalCustomerAccount.php` is invoked. This function allows the licensee to implement their own login workflow and by pass the standard Taopix login pages. If the login is successful, the user will gain access to the part of the Taopix system they were accessing. Even though the user is not logged into the Taopix system directly a user account will need to be created within Taopix Control Centre and a Taopix session will be started. As regards creating a Taopix user account, this follows similar rules as described earlier in this document for the `login` function.

```
static function ssoLogin($pParamArray)
{

        $resultArray = array();
        $result = '';
        $resultParam = '';

        $updateAccountDetails = false;
        $updateAccountBalance = false;
        $updateGiftcardBalance = false;
        $updateGroupCode = false;

        $userAccountArray = $pParamArray['useraccount'];
        $ssoToken = $pParamArray['ssotoken'];
        $ssoStage = $pParamArray['ssostage'];
        $ssoPrivateData = $pParamArray['ssoprivatedata'];
        $assetServiceData = array();
        $ssoExpireDate = '';

        // perform some processing

        $resultArray['result'] = $result;
        $resultArray['resultparam'] = $resultParam;
        $resultArray['useraccount'] = $userAccountArray;
        $resultArray['updateaccountdetails'] = $updateAccountDetails;
        $resultArray['updateaccountbalance'] = $updateAccountBalance;
        $resultArray['updategiftcardbalance'] = $updateGiftcardBalance;
        $resultArray['updategroupcode'] = $updateGroupCode;
        $resultArray['ssotoken'] = $ssoToken;
        $resultArray['ssoprivatedata'] = $ssoPrivateData;
        $resultArray['assetservicedata'] = $assetServiceData;
        $resultArray['ssoexpiredate'] = $ssoExpireDate;

        return $resultArray;

}
```

Input Parameters

The input parameter array keys provided to the script are as following:

languagecode	varchar(50)	The Taopix language code
designergroupcode	varchar(50)	The code for the Taopix License Key that was supplied to Taopix Control Centre by Taopix Designer.
brandcode	varchar(50)	The code for the Taopix brand that the user is logging in from.
isordersession	integer	0 = User is logging into their account 1 = User is logging in to place an order
reason	integer	The reason the ssoLogin function has been called. See the associated list for descriptions for each reason.
sourceurl	varchar(200)	The Taopix URL which is being called when ssoLogin has been invoked.
ssostage	integer	1 = The first time this function has been invoked for the SSO workflow 2 = The second time this function has been invoked. This will be from the callback for the licensees system if redirected.
ssotoken	varchar(200)	The token provided by the licensee which is associated to with their login workflow.
ssoprivatedata	array	A array of data the licensee can use to store persistent data regarding the login workflow integration.

useraccount	array	A PHP associate array that contains the default settings for a new Taopix user account.

Single Sign-On Reasons

The `ssoLogin` function is invoked in various different parts of the Taopix system. So that the licensee knows why the function has been invoked we provide a reason which lets them know why the function has been called. From the reason the licensee can then make a decision on whether or not to implement their login workflow or which part of the login workflow you wish to implement. For example: When *TPX_USER_AUTH_REASON_WEB_LOGIN* is passed you may wish to redirect the user to your login page and get them to sign in. But when *TPX_USER_AUTH_REASON_ONLINE_PROJECT_CREATE* is passed you may wish to perform a server to server call and check that your *ssotoken* is valid.

There are a set of constants provided by Taopix, starting with "*TPX_USER_AUTH_REASON*", which help the licensee make their decision easier. Reasons which start with "*TPX_USER_AUTH_REASON_HIGHLEVEL*" are only used when the Multiline Basket API is switched on. The Multiline Basket API documentation explains how this should be implemented.

The following table explains what Taopix functionality each reason relates to:

TPX_USER_AUTH_REASON_WEB_INIT (0)	The user has attempted to access Control Centre Login Page or the Shopping Cart from Desktop
TPX_USER_AUTH_REASON_WEB_LOGIN (1)	The action the user is attempting requires authentication
TPX_USER_AUTH_REASON_WEB_CART_INIT (2)	The user is accessing the shopping cart for the first time
TPX_USER_AUTH_REASON_WEB_CUSTOMER_SHARE_PREVIEW (3)	Once a user's project has been uploaded to production an email could be sent to them with a link to a preview of the project. On navigating to this link they will need to be authenticated
TPX_USER_AUTH_REASON_ONLINE_PROJECT_CREATE (4)	The user is attempting to create an online project
TPX_USER_AUTH_REASON_ONLINE_PROJECT_EDIT (5)	The user is attempting to edit an online project
TPX_USER_AUTH_REASON_HIGHLEVEL_CHECK_SESSION (9)	This is called for 2 different reasons: 1. The product selector is configured to attempt to log the user when the web page is loaded. 2. The user is logged into the Multiline Basket API product selector already but we need to make sure the session details are still valid
TPX_USER_AUTH_REASON_HIGHLEVEL_ONLINE_PROJECT_CREATE (10)	The user is attempting to create a project from the Multiline Basket API
TPX_USER_AUTH_REASON_HIGHLEVEL_ONLINE_PROJECT_EDIT (11)	The user is attempting to edit a project from the Multiline Basket API
TPX_USER_AUTH_REASON_HIGHLEVEL_LOGIN (12)	The user is login in from the Multiline Basket API when they initiate a login/signin action i.e. button press
TPX_USER_AUTH_REASON_HIGHLEVEL_CHECKOUT (13)	The user is checking out the items in their basket and initialising the shopping cart
TPX_USER_AUTH_REASON_HIGHLEVEL_ONLINE_PROJECT_CONTINUE_EDIT (14)	The user is either editing an online project which is already in production or they have duplicated an online project

Returning The Login Result Back To Taopix

Returning the login result back to Taopix involves the following:

1. Returning a result string which is either:
    a. An empty string for a successful authentication. It is expected the *useraccount* variable to be populated with user details.

    b. The text *SSOREDIRECT* if the single sign-on workflow involves redirecting to another site for performing the login.

c. The text *NOTHANDLED* if the script did not process the single sign-on request and Taopix should handle the login as if the script did not execute.

d. An error message that is reported back to the user. The error message can be either:
   i. A Taopix string constant that resides in either the # *Common* or *[Login]* sections of the Taopix language string files.
   ii. A error message translated into the appropriate language based on the *languagecode*.

2. Returning the *resultparam* which is either:
   a. The URL which the user needs redirecting to if the *result* is set to *SSOREDIRECT*.
   b. An extra error information if an error is being returned in the result. This needs to be translated into the appropriate language based on the *languagecode*.

3. A copy of the input user account array that has the relevant fields filled in via the script. The array key values are not relevant if the authentication is not successful. This is only populated when the *result* field is empty.

4. A flag that specifies if the account balance / credit limit supplied by the script should override the values currently held within the Taopix database customer account if the authentication is successful. This is only populated when the *result* field is empty.

5. A flag that specifies if the gift card balance supplied by the script should override the values currently held within the Taopix database customer account if the authentication is successful. This is only populated when the *result* field is empty.

6. A flag that specifies if the user account has been updated or not. If it has been updated the account with the *accountcode* from the *useraccount* field will be updated with the values in the *useraccount* field. This is only populated when the *result* field is empty.

7. A flag that specifies if the users group code has changed. If this is set the group code will be updated in the database with the one in the *useraccount* field. This will only done of the *updateaccountdetails* flag is set. This is only populated when the *result* filed is empty.

8. A token string value which has used to track the single sign-on session with the external system. This is only populated when the *result* field is empty.

9. An array of data which has come from the external system as part of the single sign-on processes. This is only populated when the *result* field is empty.

10. And array of asset service data which will be used to populate a new online project with images. You will only populate this array when the reason is set to *TPX_USER_AUTH_REASON_HIGHLEVEL_ONLINE_PROJECT_CREATE* or *TPX_USER_AUTH_REASON_ONLINE_PROJECT_CREATE.* This is only populated when the *result* field is empty.

Output Parameters

result	string	Empty = The user is valid. The useraccount field is expected to be populated. NOTHANDLED = SSO has not being invoked. SSOREDIRECT = To invoke the licensee workflow the system needs to redirect to a different URL. The URL needs to be returned in the resultparam variable. Taopix language string constant from either the # *Common* or *[Login]* sections of the Taopix language string files.
resultparam	string	If result is SSOREDIRECT then this variable must contain the URL to direct to.

		If you are using result to return an error, you can use this variable to provide more information.
useraccount	array	The modified input useraccount array that contains the settings for the Taopix user account (see below).
updateaccountbalance	boolean	false = Do not update the credit limit / account balance values currently held in the Taopix database user account with the values returned by the script. true = Replace the credit limit / account balance values currently held in the Taopix database user account with the values returned by the script.
updategiftcardbalance	boolean	false = Do not update the gift card balance value currently held in the Taopix database user account with the value returned by the script. true = Replace the gift card balance value currently held in the Taopix database user account with the value returned by the script.
updateaccountdetails	boolean	false = Do not update the user account details true = Update the user account details provided in the useraccount array
updategroupcode	boolean	false = Ignore the groupcode in the useraccount array and use the one in the database for the user true = Update the database with the groupcode in the useraccount array.
ssotoken	string	A token value from the single sign-on system which is exposed publically
ssoprivatedata	array	An array of private data from the single sign-on system which is not exposed publically.
ssoexpiredate	string	Leave empty.
assetservicedata	array	An array of asset services along with assets to pre-populate the online project with (see below).

The user account array contains the following data:

groupcode	varchar(50)	M	The code for a valid Taopix License Key that should be assigned to the user account. **NOTE** If you are allowing customers to sign-in to their account within Taopix when not placing an order this value cannot be retrieved from the *designergroupcode* input value and must be sourced from somewhere else. If the only source for the group code is Taopix Designer then the script must prevent the ability to sign-in when *designergroupcode* is empty.
accountcode	varchar(50)	M	The account code for the external user account
login	varchar(50)	M	The login for the external user account
companyname	varchar(200)	A	Company name
address1	varchar(200)	A	Address line 1
address2	varchar(200)	A	Address line 2
address3	varchar(200)	A	Address line 3
address4	varchar(200)	A	Address line 4
city	varchar(200)	A	City
county	varchar(50)	A	County
state	varchar(200)	A	State
regioncode	varchar(20)	A	Region code
region	varchar(10)	A	Region field code
postcode	varchar(200)	A	Postcode / Zipcode
countrycode	varchar(10)	M	The *iso2* country code that matches a value within the Taopix *COUNTRIES* database table.
countryname	varchar(50)	M	The name of the country that matches the *iso2* country code.
telephonenumber	varchar(50)	M	Telephone number
emailaddress	varchar(50)	M	Email address
contactfirstname	varchar(200)	M	Contact first name
contactlastname	varchar(200)	M	Contact last name

registeredtaxnumbertype	integer	A	The tax number type for the customer. Values greater than 0 are currently only supported for addresses in Brazil. 0 = Not applicable 1 = Personal 2 = Corporate
registeredtaxnumber	varchar(50)	A	The tax number in the format specified by *registeredtaxnumbertype* when *registeredtaxnumbertype* is not 0.
usedefaultpaymentmethods	integer	M	0 = Use custom set of payment methods 1 = Use default payment methods
paymentmethods	varchar(100)	C	A comma separated list of Taopix payment method codes when using *usedefaultpaymentmethods* is 0.
taxcode	varchar(20)	O	A valid Taopix Control Centre tax rate code if applying a custom product tax rate to the customer account or empty if Taopix should determine the product tax rate.
shippingtaxcode	varchar(20)	O	A valid Taopix Control Centre tax rate code if applying a custom shipping tax rate to the customer account or empty if Taopix should determine the shipping tax rate.
defaultaddresscontrol	integer	C	0 = The address / email settings are customised for this user account. 1 = The address / email settings are controlled by the license key.
uselicensekeyforshippingaddress	integer	C	0 = The user account address is used for the shipping address. 1 = The address held in the license key is used for the shipping address. **NOTE** This option only applies if *defaultaddresscontrol* is 0.
canmodifyshippingaddress	integer	C	0 = The shipping address cannot be modified. 1 = The shipping address can be modified. **NOTE** This option only applies if *defaultaddresscontrol* is 0.
canmodifyshippingcontactdetails	integer	C	0 = The shipping address contact details cannot be modified. 1 = The shipping address contact details can be modified. **NOTE** This option only applies if *defaultaddresscontrol* is 0 and *canmodifyshippingaddress* is 0.
uselicensekeyforbillingaddress	integer	C	0 = The user account address is used for the billing address. 1 = The address held in the license key is used for the billing address. **NOTE** This option only applies if *defaultaddresscontrol* is 0.
canmodifybillingaddress	integer	C	0 = The billing address cannot be modified. 1 = The billing address can be modified. **NOTE** This option only applies if *defaultaddresscontrol* is 0.
useremaildestination	integer	C	0 = Send confirmation emails to order billing address. 1 = Send confirmation emails to order shipping address.

			2 = Send confirmation emails to order shipping address and bcc order billing address. 3 = Send confirmation emails to order billing address and bcc order shipping address. **NOTE** This option only applies if *defaultaddresscontrol* is 0.
creditlimit	decimal	O	The credit limit in the currency specified by Taopix as the system currency.
accountbalance	decimal	O	The account balance in the currency specified by Taopix as the system currency.
giftcardbalance	decimal	O	The gift card balance in the currency specified by Taopix as the system currency.
sendmarketinginfo	integer	O	The status of the *Can Send Marketing Information* flag: 0 = No 1 = Yes
usedefaultcurrencycode	integer	O	0 = The customer record will ignore the license key currency setting and instead use the value passed in the currency code parameter. 1 = The customer record will ignore the value passed in the currency code parameter and instead use the currency set against the license key.
currencycode	varchar(50)	C	A valid Taopix Control Centre currency code if applying a custom currency to the customer account or empty if usedefaultcurrency is set to 1.
requirepasswordforsessioninactivity	boolean	O	True = Require user password for session inactivity. False = User password not required for session inactivity.
isactive	integer	M	0 = Account not active 1 = Account active

Key:

M	Mandatory
O	Optional
C	Conditional Please read the column description for additional information
A	Address field that may be mandatory. Please see the section on address formats for additional information

Asset Service Data

The *assetservicedata* field in described in the previous table needs to be utilities if you wish to implement the Image Led Workflow. Image Led workflow enables users to populate their project with pictures when it is created. This involves the users selecting some pictures in a non-Taopix system prior to starting the project. The pictures they select will need to be part of an asset management system which is connected to Taopix via the Library Link API. Currently Taopix only supports adding pictures from one asset management service. The *assetservicedata* variable is an associative array with one item called librarylink. This is an array of items with the following fields:

servicecode	string	This is the servicecode which is given to the asset service when added to the Taopix system via Taopix Creator.
privatedata	string	This is where you can specify any data which will be passed onto the asset service when it is invoked to extract the images from the asset service for inclusion in the project. In this example we have used the *ssotoken* since the asset service we are using is also uses SSO with the same system.
id	string	This can be whatever string data you need to pass to the asset management service. This is where you would add the id(s) of any pictures or a picture container which references the pictures.

An example of this array can be found in the Single Sign-On In Practice section in the `ssoLogin` example.

IMPORTANT
1. The user account array does actually contain more data than listed here. However, it is critical that you always return a modified version of the input array and do not modify any values apart from the ones listed here.

2. Customer accounts are unique across a Taopix brand.

3. It is not possible to have any customer account with the same username / login as any non-customer account.

4. The *isactive* flag has limited use when using external scripts to handle authentication. All customer login requests will be forwarded to the script no matter what is set within the Taopix user account and it is up to the script to determine if the account is active or not. If *isactive* is set to zero in the result Taopix will display the standard message stating that the user account is not active unless an error is returned by the script.

5. The *accountcode* has be set for Single Sign-On to work correctly. The account code is used as a link between the Single Sign-On system and the Taopix system.

NOTE
Failure to supply Taopix Control Centre with the correct information will result in the system not operating correctly.

Single Sign-On Callback

As stated in the previous section on Single Sign-On Login, the `ssoCallback` function is invoked on returning to Taopix after the user was redirected to an external site to perform the single sign-on login. Any information the external site needs to provide to Taopix for the single sign-on process (e.g. single sign-on token, user data) can to be sent via HTTP POST or GET to the script. It is then the licensees responsibility to store any information they later need in the second call to the `ssoLogin` function in the *ssotoken* and *ssoprivatedata* fields.

```
static function ssoCallback($pParamArray)
{
        $resultArray = array();
        $resultArray['ssotoken'] = $pParamArray['ssotoken'];
        $resultArray['ssoprivatedata'] = $pParamArray['ssoprivatedata'];
        $resultArray['result'] = '';
        $resultArray['resultparam'] = '';

        return $resultArray;
}
```

Input Parameters

The input parameter array keys provided to the script are as following:

ssotoken	varchar(200)	The ssotoken from any subsequent sso activity for this session.
ssoprivatedata	array	The ssoprivatedata from any subsequent sso activity for this session.

Processing A Callback Event

Setting the *ssotoken* and *ssoprivatedata* values depends on the data which is being passed from the external system however it is important that an *ssotoken* value is set. The *ssotoken* is used as an indicator that a Single Sign-On workflow has been used. The *result* and *resultparam* values are to be used to report any error which have occurred.

Returning Callback Result Back To Taopix

Returning the callback event result back to Taopix involves the following:

1. Populate the *ssotoken* string with a token which represents this single sign-on session with the external system
2. Populate the *ssoprivatedata* array with any appropriate data passed to the script via an HTTP POST or GET.
3. A Taopix string constant that resides in either the # *Common* or *[Login]* sections of the Taopix language string files.
4. Any other error text which needs to be displayed with the error.

Output Parameters

result	string	Taopix language string constant from either the # *Common* or *[Login]* sections of the Taopix language string files.
resultparam	string	If you are using result to return an error, you can use this variable to provide more information.
ssotoken	string	The token provided by the licensee which is associated to with their login workflow.
ssoprivatedata	array	A array of data the licensee can use to store persistent data regarding the login workflow integration.

Single Sign-On Default Group Code

When a user reaches Taopix you may wish to specify a default group code in the case that one hasn't been detected. This will be an issue when creating a user since taopix needs to know the group code so that any new users created can be created with the correct default settings from the license key. To get round this problem Taopix allows for you to specify a default group code for the brand which has been detected for the Single Sign-On call.

```
static function ssoGetBrandDefaultGroupCode($pParamArray)
{
        return "";
}
```

Input Parameters

The input parameter array keys provided to the script are as following:

brandcode	varchar(50)	The brand code which has been detected from URL the user navigated to.

Returning Default Group Code Result Back To Taopix

This function requires the group code you elect to use for the given brand code to be returned as a string. This group code will be used for the rest of the Single Sign-On call.

Single Sign-On Logging Out

When a user logs out of or is logged out from Taopix Control Centre or Online Designer, if they have logged in via Single Sign-On Taopix will invoke a logout function specifically for the Single Sign-On logout called `ssoLogout`. A reason, different to the ones used for logging in, will be provided to this function to identify what caused the logout, and based on that reason a logout process can be implement. You also have the option to return a URL which the user will be redirected to was the Taopix logout process has completed.

```
static function ssoLogout($pParamArray)
{
        return "";
}
```

NOTE
The logout will only log the user out of the session they are logged into where the logout occurred. This means that if the user is logged into 2 sessions and they log out of one of the them they are not logged out of both. If the user was logged into Taopix using the same session in the Single Sign-On system for both Taopix sessions, if you log the user out of the Single Sign-On session in the `ssoLogout` function or in the resulting URL, the other Taopix session would potentially still be logged in with an invalid session.

Input Parameters

The input parameter array keys provided to the script are as following:

ssotoken	varchar(200)	The ssotoken from any subsequent SSO activity for this session.
ssoprivatedata	array	The ssoprivatedata from any subsequent SSO activity for this session.
reason	integer	The reason the ssoLogout function has been called. See the associated table for descriptions for each reason.

The following table explains what Taopix functionality each logout reason relates to:

TPX_USER_LOGOUT_REASON_USER_LOGOUT (0)	The user has logged out of Taopix Control Centre or Taopix Online Designer and the login was not started from the Multiline Basket API.
TPX_USER_LOGOUT_REASON_SESSION_EXPIRED (1)	The user session has expired so the user has been automatically logged out when they attempted to perform an action.
TPX_USER_LOGOUT_REASON_SESSION_NOT_STARTED (2)	When the user attempted to start a session it failed and thus the user has been logged out.
TPX_USER_LOGOUT_REASON_HIGHLEVEL_DESIGNER (3)	The user has logged out of Taopix Online Designer and the project was started from the Multiline Basket API.
TPX_USER_LOGOUT_REASON_HIGHLEVEL_ACCOUNTPAGES (4)	The user has logged out of Taopix Control Centre and the login came from the Multiline Basket API.

Returning The Logout URL Back To Taopix

The value which is returned from this function must be a valid URL which the user will be directed to. This URL can perform the logout or a server to server call can be made within this function. If there isn't a URL to redirect the user to return an empty string. The only valid values are a valid URL and an empty string.

NOTE

If the reason passed into `ssoLogout` is *TPX_USER_LOGUT_REASON_HIGHLEVEL_DESIGNER* or *TPX_USER_LOGUT_REASON_HIGHLEVEL_ACCOUNTPAGES*, and the user will ultimately be returning back to the product selector or shop front they came from. So that the Multiline Basket API deletes the basket correctly it is important the page which is using the Multiline Basket API has the parmaters `odlo=1` appended to it. Failing to do so could cause users baskets to not be emptied correctly.

Single Sign-On Setting The Home URL In An Email

When an email is sent from Taopix to a user (e.g. upload complete email) there is a link appended to the end of the email which directs the user back into Taopix. The callback into Taopix could invoke the `ssoLogin` function and potentially this function could be looking for extra URL parameters as part of the Single Sign-On workflow. Taopix provide a function which allows the link contained within emails to be modified to include any extra parameters which may be needed.

```
static function ssoSetHomeURL($pParamArray)
{
        return $pParamArray['url'];
}
```

Input Parameters

The input parameter array keys provided to the script are as following:

ssotoken	varchar(200)	The ssotoken from any subsequent SSO activity for this session.
ssoprivatedata	array	The ssoprivatedata from any subsequent SSO activity for this session.
brandcode	string	The brand code associated to the email which is being sent.
url	string	The original URL which is being sent in the email

Returning Default Group Code Result Back To Taopix

This function requires a URL to be returned from the function. This is either the modified email or the original. Failing to return a valid URL will result in the link on the email failing.

Single Sign-On In Practice

As stated previously, it is difficult for Taopix to provide any sample code to aide implementing Single Sign-On since the way external systems work can differ dramatically. Instead, in this section we attempt to provide ways in which each function could be implemented explaining the scenario in which they are intended.

It is important that you have read the rest of the section on Single Sign-On before attempting the function implementations below.

ssoGetBrandDefaultGroupCode

```
static function ssoGetBrandDefaultGroupCode($pParamArray)
{

        $groupCode = "";

        switch ($pParamArray['brandcode'])
        {
                case "BRANDA":
                {
                        $groupCode = "GROUPA";
                        break;
                }
                case "BRANDB":
                {
                        $groupCode = "GROUPB";
                        break;
                }
        }

        return $groupCode;
}
```

Basically this implementation will check the brand code and set the group code accordingly to it's value. The group code which is being returned should be an invalid group code in the Taopix system.

ssoLogin

```
        static function ssoLogin($pParamArray)
        {

                $resultArray = array();
                $result = '';
                $resultParam = '';

                $updateAccountDetails = false;
                $updateAccountBalance = false;
                $updateGiftcardBalance = false;
                $updateGroupCode = false;

                $userAccountArray = $pParamArray['useraccount'];
                $ssoToken = $pParamArray['ssotoken'];
                $ssoStage = $pParamArray['ssostage'];
                $ssoPrivateData = $pParamArray['ssoprivatedata'];
                $assetServiceData = array();
                $ssoExpireDate = '';

                // perform some processing

                // if the parameter nosso is present in the GET query string do not perform sso
                if (isset($_GET['nosso']))
                {
                        $result = 'NOTHANDLED';
                }
                else
                {
                        switch($ssoStage)
                        {
                                case 1:
                                {
                                        // the SSO stage is set to 1 which means this is the first time
the ssoLogin function has been called for this action. we need to redirect the user to our single sign-on
system login page
                                        $result = 'SSOREDIRECT';
                                        $resultParam = "http://www.ssosystem.com/login/";

                                        break;
                                }
                                case 2:
                                {

                                        // ths SSO stage is set to 2 which means this is the second time
the ssoLogin function has been called for this action.
                                        // the ssoprivatedata variable has been populated with the
details from the login workflow which we need to populate the Taopux user account

                                        $userAccountArray['login'] = $ssoPrivateData['username'];
                                        $userAccountArray['password'] = md5($ssoPrivateData['username'] .
$ssoPrivateData['id']);
                                        $userAccountArray['accountcode'] =
$ssoPrivateData['accountcode'];
                                        $userAccountArray['contactfirstname'] =
$ssoPrivateData['firstname'];
                                        $userAccountArray['contactlastname'] =
$ssoPrivateData['lastname'];
                                        $userAccountArray['countrycode'] =
$ssoPrivateData['countrycode'];
                                        $userAccountArray['countryname'] =
$ssoPrivateData['countryname'];
                                        $userAccountArray['emailaddress'] =
$ssoPrivateData['emailaddress'];
                                        $userAccountArray['giftcardbalance'] =
$ssoPrivateData['giftcardbalance'];
                                        $userAccountArray['creditlimit'] =
$ssoPrivateData['creditlimit'];
                                        $userAccountArray['accountbalance'] =
$ssoPrivateData['accountbalance'];
                                        $userAccountArray['isactive'] = 1;

                                        $updateAccountDetails = true;
                                        $updateAccountBalance = false;
                                        $updateGiftCardBalance = false;
                                        $updateGroupCode = false;

                                        // set the ssoToken based on the token which has been returned
from the sso system
                                        $ssoToken = $ssoPrivateData['token'];

                                        // if the reason this function has been called is becasue the
user wishes to create an online project
                                        // and a list of images have been added to the URL when the call
was made, set up the librarylink variable
                                        // with the appropriate data
                                        if (($pParamArray['reason'] ==
TPX_USER_AUTH_REASON_ONLINE_PROJECT_CREATE) ||
```

```
                                                        ($pParamArray['reason'] ==
TPX_USER_AUTH_REASON_HIGHLEVEL_ONLINE_PROJECT_CREATE))
                                                {
                                                        if (isset($_GET['imagelist']))
                                                        {
                                                                $assetServiceData = array('librarylink' =>
array());

                                                                $assetServiceData['librarylink'][] =
array('servicecode' => 'GETTY',

                                                        'privatedata' => $ssoToken,

                                                        'id' => $_GET['imagelist']);
                                                        }
                                                }

                                                break;
                                        }

                                }

                        }

                        $resultArray['result'] = $result;
                        $resultArray['resultparam'] = $resultParam;
                        $resultArray['useraccount'] = $userAccountArray;
                        $resultArray['updateaccountdetails'] = $updateAccountDetails;
                        $resultArray['updateaccountbalance'] = $updateAccountBalance;
                        $resultArray['updategiftcardbalance'] = $updateGiftcardBalance;
                        $resultArray['updategroupcode'] = $updateGroupCode;
                        $resultArray['ssotoken'] = $ssoToken;
                        $resultArray['ssoprivatedata'] = $ssoPrivateData;
                        $resultArray['assetservicedata'] = $assetServiceData;
                        $resultArray['ssoexpiredate'] = $ssoExpireDate;

                        return $resultArray;

                }
```

This is an example of a system which redirects their users to an external site to perform the Single Sign-On workflow. The first thing to notice in this example is the usage of checking the query string to control the workflow. The first thing which the script does is check for a parameter on the query string call *nosso*. This is a non-Taopix parameter which we have included in this example to demonstrate how you could switch SSO off. The code checks for this parameter and if it is present it will not perform any SSO workflow. This is useful if you need to access the administrator side of Taopix Control Centre. Without this parameter on the URL the user will be redirected. e.g:

This will take the user to the Single Sign-On workflow site:

`http://control.joebloggsphotos.com/`

This will display the Taopix Control Centre login:

`http://control.joebloggsphotos.com?nosso=1`

Once the script has established it is performing the Single Sign-On request it will check to see what stage of the workflow we are at.

If we are at stage 1 it will return *SSOREDIRECT* as it's result and the URL of the external system as it's *resultparam*. This will result in the user being redirected to the URL in the *resultparam*.

If we are at stage 2 we can now populate the *useraccount* variable with any details which have been fed into the system from the external Single Sign-On system. For this example, this has been done in the `ssoCallback` function using the *ssoprivatedata* variable. The callback script will have been posted the values and when the `ssoCallback` function was called it will copy the values into the *ssoprivatedata* variable.

After the data is copied from the *ssoprivatedata* to the *useraccount* variable we then check the reason. If the `ssoLogin` has been called because the user is creating a new online project, we will look at the query string and pull out the non-Taopix *imagelist* variable. We then use this variable to setup the asset service data for the Image Led workflow.

ssoCallback

```
static function ssoCallback($pParamArray)
{
        $resultArray = array();
        $resultArray['ssotoken'] = $pParamArray['ssotoken'];
        $resultArray['ssoprivatedata'] = array('username' => $_POST["username"],'token' =>
             $_POST["token"],'id' => $_POST["id"],'firstname' => $_POST["firstname"],'lastname' =>
             $_POST["lastname"],'countrycode' => $_POST["countrycode"],'countryname' =>
             $_POST["countryname"],'emailaddress' => $_POST["emailaddress"],'creditlimit' =>
             $_POST["creditlimit"],'accountbalance' => $_POST["accountbalance"],'giftcardbalance'
             => $_POST["giftcardbalance"],'accountcode' => $_POST["accountcode"]);

        $resultArray['result'] = '';
        $resultArray['resultparam'] = '';

        return $resultArray;
}
```

As stated in the `ssoLogin` example, this function will read values which are posted to the callback script and assign them to the *ssoprivatedata* variable and then in the `ssoLogin` function these values are read and used to populate the user account. In this example all the user's details are sent using an HTTP POST. Another way of implementing this function could be to only POST an id which you then use in a server to server callback to the SSO system to grab the data in an encrypted manner.

ssoSetHomeURL

```
public static function ssoSetHomeURL($pParamArray)
{
        $extraURLParameters = array();
        $returnURL = $pParamArray['url'];

        if ($pParamArray['ssoprivatedata']['accesstoken'] != '')
        {
                $extraURLParameters[] = array('accesstoken' =>
                                $pParamArray['ssoprivatedata']['accesstoken']);
        }

        if ($pParamArray['brandcode'] == "BRANDA")
        {
                $extraURLParameters[] = array('groupcode' => "GROUPA");
        }

        $count = count($extraURLParameters);
        $index = 0;

        for ($i = 0; $i < $count; $i++)
        {
                foreach ($extraURLParameters[$i] as $key => $value)
                {
                        $connector = "&";

                        if ($i == 0)
                        {
                                if (strrpos($returnURL, "?") === false)
                                {
                                        $connector = "?";
                                }
                        }

                        $returnURL .= $connector . $key . "=" . $value;
                }
        }

        return $returnURL;
}
```

This function is adding extra parameters on to the URL which will appear in the email based on values which have been supplied. In this example we are looking in the *ssoprivatedata* array for the *accesstoken* and appending it the URL if found. The *accesstoken* variable is not a Taopix specific value and is something which has been assigned to the *ssoprivatedata* in either the `ssoLogin` or `ssoCallback` function. Like with the `ssoGetBrandDefaultGroupCode` we are checking which brand code has been supplied to the function and appending the appropriate group code to the URL.

Validating a Customer Password

If the security setting 'Confirm Account Changes' is set in Taopix Control Centre, each customer will be forced to enter their password when making a change to their account details. Taopix will divert processing to the `authenticate` function within the external PHP script `EDL_ExternalCustomerAccount.php`.

The purpose of this function is to use the parameters provided as a PHP associative array to validate a customer password within the external system and return the relevant data, also as an associative array, back to Taopix Control Centre.

```
function authenticate($pParamArray)
{
        $resultArray = [];
        $result = '';

        // perform some processing

        $resultArray['result'] = $result;
        return $resultArray;
}
```

Input Parameters

The input parameter array keys provided to the script are as following:

languagecode	varchar(50)	The Taopix language code
login	varchar(50)	The login name that was originally assigned to the customer account held within the Taopix database.
password	varchar(50)	The password in the specified format.

passwordformat	varchar(50)	The format of the password data:
		`TPX_PASSWORDFORMAT_CLEARTEXT`
		`TPX_PASSWORDFORMAT_MD5`
		
		If Taopix Control Centre is running under HTTPS / SSL and the password has been updated then the password will be in clear text otherwise it will be a MD5 hash.

designergroupcode	varchar(50)	The code for the Taopix License Key that was supplied to Taopix Control Centre by Taopix Designer.
accountgroupcode	varchar(50)	The code for the Taopix License Key that is assigned to the customer account if one already exists within the Taopix database.
brandcode	integer	The code for the Taopix Brand now assigned to the customer account within the Taopix database.
accountcode	varchar(50)	The account code now assigned to the customer account held within the Taopix database.
useraccount	array	The settings for the customer account (see below).

The user account array contains the following data:

companyname	varchar(200)	Company name (see address formats below)
address1	varchar(200)	Address line 1 (see address formats below)
address2	varchar(200)	Address line 2 (see address formats below)
address3	varchar(200)	Address line 3 (see address formats below)
address4	varchar(200)	Address line 4 (see address formats below)
city	varchar(200)	City (see address formats below)
county	varchar(50)	County (see address formats below)
state	varchar(200)	State (see address formats below)
regioncode	varchar(20)	Region code (see address formats below)

region	varchar(10)	Region field code (see address formats below)
postcode	varchar(200)	Postcode / Zipcode (see address formats below)
countrycode	varchar(10)	The *iso2* country code that matches a value within the Taopix *COUNTRIES* database table.
countryname	varchar(50)	The name of the country that matches the *iso2* country code.
telephonenumber	varchar(50)	Telephone number
emailaddress	varchar(50)	Email address
contactfirstname	varchar(200)	Contact first name
contactlastname	varchar(200)	Contact last name
registeredtaxnumbertype	integer	The tax number type for the customer. Values greater than 0 are currently only supported for addresses in Brazil. 0 = Not applicable 1 = Personal 2 = Corporate
registeredtaxnumber	varchar(50)	The tax number in the format specified by *registeredtaxnumbertype* when *registeredtaxnumbertype* is not 0.
usedefaultpaymentmethods	integer	0 = Use custom set of payment methods 1 = Use default payment methods
paymentmethods	varchar(100)	A comma separated list of Taopix payment method codes when using *usedefaultpaymentmethods* is 0.
taxcode	varchar(20)	A valid Taopix Control Centre tax rate code if applying a custom product tax rate to the customer account or empty if Taopix should determine the product tax rate.
shippingtaxcode	varchar(20)	A valid Taopix Control Centre tax rate code if applying a custom shipping tax rate to the customer account or empty if Taopix should determine the shipping tax rate.
defaultaddresscontrol	integer	0 = The address / email settings are customised for this user account. 1 = The address / email settings are controlled by the license key.
uselicensekeyforshippingaddress	integer	0 = The user account address is used for the shipping address. 1 = The address held in the license key is used for the shipping address. **NOTE** This option only applies if *defaultaddresscontrol* is 0.
canmodifyshippingaddress	integer	0 = The shipping address cannot be modified. 1 = The shipping address can be modified. **NOTE** This option only applies if *defaultaddresscontrol* is 0.
canmodifyshippingcontactdetails	integer	0 = The shipping address contact details cannot be modified. 1 = The shipping address contact details can be modified. **NOTE** This option only applies if *defaultaddresscontrol* is 0 and *canmodifyshippingaddress* is 0.
uselicensekeyforbillingaddress	integer	0 = The user account address is used for the billing address. 1 = The address held in the license key is used for the billing address. **NOTE** This option only applies if *defaultaddresscontrol* is 0.
canmodifybillingaddress	integer	0 = The billing address cannot be modified.

		1 = The billing address can be modified. **NOTE** This option only applies if *defaultaddresscontrol* is 0.
useremaildestination	integer	0 = Send confirmation emails to order billing address. 1 = Send confirmation emails to order shipping address. 2 = Send confirmation emails to order shipping address and bcc order billing address. 3 = Send confirmation emails to order billing address and bcc order shipping address. **NOTE** This option only applies if *defaultaddresscontrol* is 0.
canmodifypassword	integer	0 = The password cannot be modified by the user. 1 = The password can be modified by the user.
creditlimit	decimal	The credit limit in the currency specified by Taopix as the system currency.
accountbalancedifference	decimal	The account balance difference in the currency specified by Taopix as the system currency.
giftcardbalancedifference	decimal	The gift card balance difference in the currency specified by Taopix as the system currency.
sendmarketinginfo	integer	The status of the *Can Send Marketing Information* flag: 0 = No 1 = Yes
isactive	integer	0 = Account not active 1 = Account active

Processing a Password Validation Request

The actual task of validating the password will depend upon the external system responsible for the customer accounts but it will probably involve checking the brand code and password supplied by Taopix and returning whether or not the validation was successful.

Returning the Result Back to Taopix

The function requires that an associative array with a single key (see below) is returned. The result should either be an empty string or the name of the Taopix language string of an error.

Output Parameters

result	string	Empty or a Taopix language string constant from either the # *Common* section of the Taopix language string files.

Taopix Address Formats

The Taopix system supports mailing addresses belonging to different countries that may have a different number of fields and validation requirements. When supplying the address information in the user account array the data must be in the correct format for the country specified. The reason for this is that even though the customer account has originated from an external system it will actually be inserted into the Taopix database as a standard customer account so that Taopix can retrieve account data as required.

The address formats for each country are held within the *COUNTRIES*, *COUNTRYREGION* and *COUNTRYREGIONGROUPS* database tables of the Taopix database.

COUNTRIES Database Table

As well as holding the ISO codes and names of countries supported by Taopix it also holds information on the structure of the address.

Within the *COUNTRIES* database table the columns *region*, *displayfields*, *compulsoryfields* and *fieldlabels* hold the important information on the structure of an address.

region

The *region* column specifies if the country is broken down into regions and will either be an empty string for no regions, STATE for the state field being a region and COUNTY for county field being a region. When a country is specified as having regions the standard Taopix address form input changes from an entry field to a dropdown list that contains values from the *COUNTRYREGION* table. Each region has a code and a name that is used to populate the *region*, *regioncode* and the relevant field (*state* / *county*) within the customer account.

displayfields

The *displayfields* column lists the fields that appear for a specific country when editing addresses within Taopix. The data is held as a list of field names separated by the tag *<p>*.

Examples:
1. The address format for the UK is:
```
country<p>firstname<p>lastname<p>company<p>add1<p>add2<p>add3<p>add4<p>city<p>county
<p>postcode
```

2. The address format for the USA is:
```
country<p>firstname<p>lastname<p>company<p>add1<p>add2<p>city<p>state<p>county<p>postcode
```

If the *displayfields* column is empty then Taopix defaults to a standard set of fields which is:
```
country<p>firstname<p>lastname<p>company<p>add1<p>add2<p>add3<p>add4<p>city<p>county<p>state
<p>postcode
```

compulsoryfields

The *compulsoryfields* column lists the fields that must be completed when editing addresses within Taopix. The data is held as a list of field names separated by a comma.

Examples:
1. The compulsory fields for the UK is:
```
country,firstname,lastname,add1,city,county,postcode
```

2. The compulsory fields for the USA is:
```
country,firstname,lastname,add1,city,state,postcode
```

If the *compulsoryfields* column is empty Taopix defaults to a standard set of fields which is:
```
country,firstname,lastname,add1,city,postcode
```

fieldlabels

The fieldlabels column lists the labels used for each display field when editing addresses within Taopix. Although this column is not required in the user account array to contents may help you identify what the fields are used for. The data is held as a list of Taopix language strings separated by a comma.

Examples:
1. The field labels for the UK is:
```
str_LabelCountry,str_LabelFirstName,str_LabelLastName, str_LabelLastName,
str_LabelLastName,str_LabelCompanyName,str_LabelAddressLine1,str_LabelAddressLine2,str_LabelAd
dressLine3,str_LabelAddressLine4,str_LabelTownCity,str_LabelCounty,str_LabelPostCode
```

2. The field labels for the USA is:
```
str_LabelCountry,str_LabelFirstName,str_LabelLastName,str_LabelCompanyName,str_LabelAddressLin
e1,str_LabelAddressLine2,str_LabelTownCity,str_LabelState,str_LabelCounty,str_LabelZIPCode
```

If the *fieldlabels* column is empty then Taopix defaults to a standard set of fields which is:
```
str_LabelCountry,str_LabelFirstName,str_LabelLastName,str_LabelCompanyName,str_LabelAddressLin
e1,str_LabelAddressLine2,str_LabelAddressLine3,str_LabelAddressLine4,str_LabelTownCity,str_Lab
elCounty,str_LabelState,str_LabelPostCodee
```

COUNTRYREGION Database Table

The *COUNTRYREGION* database table contains records for each country that has regions enabled. Within the database table the columns *countrycode*, *regioncode*, *regionname* and *regiongroup* hold the important information for an individual region.

countrycode

The *countrycode* column contains the ISO2 country code that the region entry relates to.

regioncode

The *regioncode* column contains the unique code for the region entry.

regionname

The *regionname* column contains the region name (eg: county name / state name). The data in this column is stored using the Taopix multi-lingual string format but is not stored in the user account in multi-lingual format.

regiongroupcode

When Taopix displays the region dropdown list it is possible to group regions together (ie: regions for England, Scotland, Wales, Northern Ireland). When regions are grouped this column will hold the group code that relates to the group held in the *COUNTRYREGIONGROUPS* database table.

COUNTRYREGIONGROUPS Database Table

The *COUNTRYREGIONGROUPS* database table contains information on how each country's group of regions should be presented. Further knowledge of this data is not required for the purposes of the scripting as it is only used when Taopix displays it's dynamic address form.

Taopix Multi-Lingual String Format

Taopix stores various names and descriptions for multiple languages as a single string in a format known as the Taopix multi-lingual string format. The format of a multi-lingual string is the language code followed by a space and then the text for the language. Strings that contain multiple languages are then separated by the paragraph <*p*> tag.

Examples:

```
en this is the English description

en this is the English description<p>de this is the German description
```

Limitations

The following limitations have been identified when using external customer account scripting.

1. If the external system does not store its passwords as an md5 hash or has no way to retrieve the original clear text password and convert it to a md5 hash then Taopix Web must be run under HTTPS / SSL. This is required so that Taopix can use the browser security to transmit the login information to Taopix Control Centre as clear text which can then be converted by the external script into the same password format that the external system requires.

2. Licensees who use the External Shopping Cart API will not receive the *login* API call when logging in to place an order within the external shopping cart. The *updateAccountBalance* and *updateGiftCardBalance* API calls will also not be received when using an external shopping cart. If a customer is allowed to login to their account via Taopix then the script will still receive all API calls for actions that can be triggered by a customer when logging into their account and whilst in their account.

3. You should try to avoid re-assigning customer accounts to different Taopix Brands or License Key codes once they have been inserted into the Taopix database for the first time as doing this may result in undocumented behaviour.

4. Modifying customer account records directly within the database will not trigger the external customer account script.

5. Not all functions for External Customer Account can be used in conjunction with Single Sign-On due to the relevant Single Sign-On data not being provided to them. These functions are explicitly mentioned in this document.